



Assignment No: 2

Name : Jawad Ali

Registration No: BF25NWELE0691

Section : B

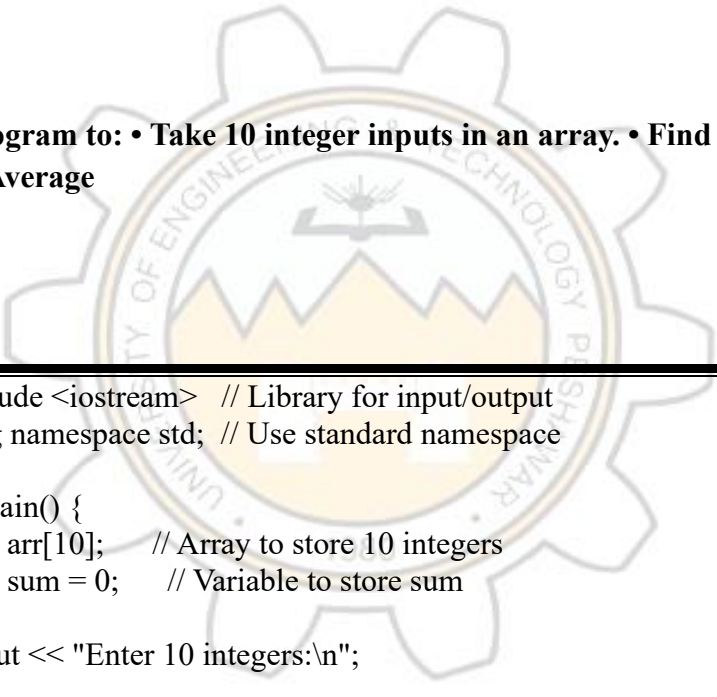
Department : Electrical (AI)

Submitted to: Rizwan sir

*** Subject Title: COMPUTER PROGRAMMING**

Program 1:

Write a program to: • Take 10 integer inputs in an array. • Find and display: Maximum value Minimum value Average



```
#include <iostream> // Library for input/output
using namespace std; // Use standard namespace

int main() {
    int arr[10]; // Array to store 10 integers
    int sum = 0; // Variable to store sum

    cout << "Enter 10 integers:\n";

    // Loop to take input
    for(int i = 0; i < 10; i++) {
        cin >> arr[i]; // Store value in array
        sum += arr[i]; // Add value to sum
    }

    int max = arr[0]; // Assume first value is max
    int min = arr[0]; // Assume first value is min

    // Loop to find max and min
    for(int i = 1; i < 10; i++) {
        if(arr[i] > max) // If current value is greater
            max = arr[i]; // Update max

        if(arr[i] < min) // If current value is smaller
            min = arr[i]; // Update min
    }
```

```

    }

    float avg = sum / 10.0; // Calculate average

    cout << "Maximum = " << max << endl;
    cout << "Minimum = " << min << endl;
    cout << "Average = " << avg << endl;

    return 0; // End program
}

```

Program 2:

Write a program that:

- Uses a function to reverse a 1D array.
- Display the array before and after reversal.

```

#include <iostream>
using namespace std;

// Function to reverse array
void reverseArray(int arr[], int size) {
    // Loop runs till half of array
    for(int i = 0; i < size / 2; i++) {
        // Swap first with last
        swap(arr[i], arr[size - i - 1]);
    }
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5}; // Initialize array

    cout << "Before Reverse:\n";

    // Display original array
    for(int i = 0; i < 5; i++)
        cout << arr[i] << " ";

    reverseArray(arr, 5); // Call function

    cout << "\nAfter Reverse:\n";

    // Display reversed array
    for(int i = 0; i < 5; i++)
        cout << arr[i] << " ";

    return 0;
}

```

Program 3:

Write a program to: • Input two 3×3 matrices. • Perform matrix addition. • Display the result matrix.

```
#include <iostream>    // Include library
using namespace std;   // Use standard namespace

int main() {           // Main function
    int A[3][3], B[3][3], C[3][3]; // Declare three 3x3 matrices

    cout << "Enter first matrix:\n"; // Ask user for first matrix

    for(int i = 0; i < 3; i++)        // Outer loop for rows
        for(int j = 0; j < 3; j++)    // Inner loop for columns
            cin >> A[i][j];           // Input value for matrix A

    cout << "Enter second matrix:\n"; // Ask for second matrix

    for(int i = 0; i < 3; i++)        // Loop rows
        for(int j = 0; j < 3; j++)    // Loop columns
            cin >> B[i][j];           // Input matrix B

    for(int i = 0; i < 3; i++)        // Loop rows
        for(int j = 0; j < 3; j++)    // Loop columns
            C[i][j] = A[i][j] + B[i][j]; // Add matrices

    cout << "Result Matrix:\n";       // Print result message

    for(int i = 0; i < 3; i++) {      // Loop rows
        for(int j = 0; j < 3; j++)    // Loop columns
            cout << C[i][j] << " "; // Print each element
        cout << endl;                // Move to next line
    }

    return 0; // End program
}
```

Program 4: Write a program that: • Stores names of 5 students in a string array. • Sorts them in alphabetical order. • Displays the sorted list.

```
#include <iostream>    // Include input-output library
#include <string>       // Include string library
using namespace std;   // Use standard namespace

int main() {           // Main function starts
    string names[5];    // Declare array of 5 strings

    cout << "Enter 5 names:\n"; // Ask user for input

    for(int i = 0; i < 5; i++) { // Loop to input names
        cin >> names[i];        // Store each name
    }

    for(int i = 0; i < 5; i++) { // Outer loop for sorting
        for(int j = i + 1; j < 5; j++) { // Inner loop for comparison
            if(names[i] > names[j]) // Compare strings alphabetically
                swap(names[i], names[j]); // Swap if out of order
        }
    }

    cout << "Sorted Names:\n"; // Print heading

    for(int i = 0; i < 5; i++) { // Loop to display names
        cout << names[i] << endl; // Print each name
    }

    return 0; // End program
}
```

- **Program 5:**

Power System Load Analysis (1D Array + Function) In a power system, hourly load demand (in MW) is recorded. Write a program to: • Store load data for 24 hours in an array. • Create functions to calculate: Total load Peak

```

#include <iostream>    // Include library
using namespace std;  // Use standard namespace

int totalLoad(int arr[], int size) { // Function to calculate total
    int sum = 0;           // Initialize sum
    for(int i = 0; i < size; i++) { // Loop through array
        sum = sum + arr[i];    // Add each value
    }
    return sum;           // Return total
}

int peakLoad(int arr[], int size) { // Function to find max value
    int max = arr[0];        // Assume first value is max
    for(int i = 1; i < size; i++) { // Loop from second element
        if(arr[i] > max)      // Check if greater
            max = arr[i];    // Update max
    }
    return max;           // Return peak value
}

int main() {              // Main function
    int load[24];         // Array for 24 hours

    cout << "Enter 24 load values:\n"; // Ask for input

    for(int i = 0; i < 24; i++) { // Loop for input
        cin >> load[i];        // Store each value
    }

    cout << "Total Load = " << totalLoad(load, 24) << endl; //
    Call function
    cout << "Peak Load = " << peakLoad(load, 24) << endl; //
    Call function

    return 0; // End program
}

```

- **Program 6:**

Design a program to: Store student names (string array) and their marks (1D array). Use functions to: Find topper Display pass/fail status Search a student by name

```

#include <iostream>    // Include library
#include <string>      // Include string
using namespace std;  // Use namespace

```

```

int findTopper(int marks[], int size) { // Function to find
topper
    int index = 0;           // Assume first student is topper
    for(int i = 1; i < size; i++) { // Loop through marks
        if(marks[i] > marks[index]) // Compare marks
            index = i;           // Update index
    }
    return index;           // Return topper index
}

void searchStudent(string names[], int marks[], int size,
string key) { // Search function
    for(int i = 0; i < size; i++) { // Loop through names
        if(names[i] == key) { // If name matches
            cout << "Found: " << names[i] << " Marks: " <<
marks[i] << endl;
            return;           // Exit function
        }
    }
    cout << "Student not found\n"; // If not found
}

int main() { // Main function
    string names[5];         // Array for names
    int marks[5];           // Array for marks

    cout << "Enter names and marks:\n"; // Input message

    for(int i = 0; i < 5; i++) { // Loop for input
        cin >> names[i] >> marks[i]; // Store name and marks
    }

    int topper = findTopper(marks, 5); // Call topper function

    cout << "Topper: " << names[topper] << endl; // Display
topper

    for(int i = 0; i < 5; i++) { // Loop for pass/fail
        if(marks[i] >= 50) // Check condition
            cout << names[i] << " Pass\n";
        else
            cout << names[i] << " Fail\n";
    }

    string key;              // Variable for search
    cout << "Enter name to search: ";
    cin >> key;              // Input search name

    searchStudent(names, marks, 5, key); // Call search
function

    return 0; // End program
}

```

```
}
```

Program 7:

Renewable Energy Monitoring System (1D Array + Functions) A solar power plant records the energy generated (in kWh) every hour for one day.

Write a C++ program to:

- Store energy generation data for 24 hours in a 1D array.

Create functions to:

- Calculate total energy generated in a day
- Find the hour with maximum generation
- Calculate average energy generation
- Display all results clearly.

```
#include <iostream>    // Include library
using namespace std;    // Use namespace

float totalEnergy(float arr[], int size) { // Total energy
function
    float sum = 0;        // Initialize sum
    for(int i = 0; i < size; i++) {    // Loop
        sum += arr[i];    // Add values
    }
    return sum;    // Return sum
}

int maxHour(float arr[], int size) {    // Function to
find max hour
    int index = 0;        // Assume first index
    for(int i = 1; i < size; i++) {    // Loop
        if(arr[i] > arr[index])    // Compare
            index = i;    // Update index
    }
    return index;    // Return index
}

float average(float arr[], int size) {    // Average
function
    return totalEnergy(arr, size) / size; // Return
average
}

int main() {    // Main function
```



```

float energy[24];           // Array of 24 hours

cout << "Enter 24 values:\n"; // Input message

for(int i = 0; i < 24; i++) { // Loop
    cin >> energy[i];         // Store values
}

cout << "Total = " << totalEnergy(energy, 24) << endl;
cout << "Peak Hour = " << maxHour(energy, 24) << endl;
cout << "Average = " << average(energy, 24) << endl;

return 0; // End
}

```

↩ program 8:

Battery Health Monitoring (1D Array + Functions) In battery management systems, voltage readings are monitored to ensure safe operation.

Write a program to:

- Store 20 voltage readings of a battery in an array. •
- Define safe voltage limits (e.g., 3.0V to 4.2V).
- Create functions to:
 - Count how many readings are below or above safe limits •
- Find the maximum and minimum voltage •

Display a warning message if unsafe readings exist

```

#include <iostream> // Include library
using namespace std; // Use namespace

int main() { // Main function
    float v[20]; // Array for 20 readings
    float low = 3.0, high = 4.2; // Safe limits

    int below = 0, above = 0; // Counters

    cout << "Enter 20 readings:\n";

    for(int i = 0; i < 20; i++) { // Input loop
        cin >> v[i]; // Store value
    }
}

```



```

    if(v[i] < low)        // Check below limit
        below++;        // Increment
    else if(v[i] > high)   // Check above limit
        above++;        // Increment
    }

    float max = v[0], min = v[0]; // Initialize max and min

    for(int i = 1; i < 20; i++) { // Loop
        if(v[i] > max) max = v[i]; // Update max
        if(v[i] < min) min = v[i]; // Update min
    }

    cout << "Below = " << below << endl;
    cout << "Above = " << above << endl;
    cout << "Max = " << max << endl;
    cout << "Min = " << min << endl;

    return 0; // End
}

```

program 9:

Smart Grid Load Distribution (2D Array + Functions) In a smart grid, electricity consumption is recorded across multiple regions.

Write a program to:

- Store power consumption (in MW) for 4 regions over 7 days using a 2D array.
- Create functions to:
- Calculate total consumption for each region
- Identify the day with highest total demand
- Compute overall average consumption

```

#include <iostream>
using namespace std;

// Total consumption for each region
void regionTotal(int arr[4][7])
{
    for(int i = 0; i < 4; i++)
    {
        int sum = 0;
        for(int j = 0; j < 7; j++)
        {
            sum += arr[i][j];
        }
        cout << "Region " << i+1 << " Total: " << sum
        << " MW" << endl;
    }
}

// Find day with highest demand
void highestDay(int arr[4][7])
{
    int maxDay = 0, maxSum = 0;

    for(int j = 0; j < 7; j++)
    {
        int sum = 0;
        for(int i = 0; i < 4; i++)
        {
            sum += arr[i][j];
        }

        if(sum > maxSum)
        {
            maxSum = sum;
            maxDay = j;
        }
    }

    cout << "Day with highest demand: Day " <<
    maxDay+1 << endl;
}

// Overall average
void averageConsumption(int arr[4][7])
{

```

```

int total = 0;

for(int i = 0; i < 4; i++)
{
    for(int j = 0; j < 7; j++)
    {
        total += arr[i][j];
    }
}

float avg = total / 28.0;
cout << "Overall Average Consumption: " << avg
<< " MW" << endl;
}

int main()
{
    int grid[4][7];

    cout << "Enter power consumption (4 regions x 7
days):\n";
    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < 7; j++)
        {
            cin >> grid[i][j];
        }
    }

    regionTotal(grid);
    highestDay(grid);
    averageConsumption(grid);

    return 0;
}

```

program 10:

Classroom Attendance System (2D Array + Functions) Teachers often track student attendance daily.

Write a program to:

- Store attendance for 5 students over 7 days using a 2D array (Use 1 for present, 0 for absent).
- Create functions to:

- Calculate total attendance of each student
- Find the student with highest attendance
- Calculate overall class attendance percentage

```
#include <iostream>
using namespace std;

// Total attendance of each student
void studentAttendance(int arr[5][7])
{
    for(int i = 0; i < 5; i++)
    {
        int sum = 0;
        for(int j = 0; j < 7; j++)
        {
            sum += arr[i][j];
        }
        cout << "Student " << i+1 << " Attendance: " << sum
<< endl;
    }
}

// Find student with highest attendance
void highestStudent(int arr[5][7])
{
    int max = 0, student = 0;

    for(int i = 0; i < 5; i++)
    {
        int sum = 0;
        for(int j = 0; j < 7; j++)
        {
            sum += arr[i][j];
        }

        if(sum > max)
        {
            max = sum;
            student = i;
        }
    }
}
```

```

    cout << "Highest Attendance: Student " << student+1 <<
endl;
}

// Overall class attendance percentage
void classPercentage(int arr[5][7])
{
    int total = 0;

    for(int i = 0; i < 5; i++)
    {
        for(int j = 0; j < 7; j++)
        {
            total += arr[i][j];
        }
    }

    float percentage = (total / 35.0) * 100;
    cout << "Overall Attendance Percentage: " <<
percentage << "%" << endl;
}

int main()
{
    int attendance[5][7];

    cout << "Enter attendance (1 = Present, 0 = Absent):\n";
    for(int i = 0; i < 5; i++)
    {
        for(int j = 0; j < 7; j++)
        {
            cin >> attendance[i][j];
        }
    }

    studentAttendance(attendance);
    highestStudent(attendance);
    classPercentage(attendance);

    return 0;
}

```

